Kandypack Rail & Road Distribution System — 4 Week Project Schedule

Start date: Today **Pace:** ~2–3 hrs/day this week, adjust weekly based on availability **Stack assumption:** MySQL/PostgreSQL + a backend (Node/Express or Python/Django/Flask) + React frontend — update once stack is confirmed **Scope confirmed from official brief:** Full-stack with a *basic* UI (not exhaustive). Database must be hand-written and deeply understood by you — this is the graded core. Backend and frontend can use AI assistance freely since deep manual understanding isn't required there.

WEEK 1 — Database Design & Setup (this week, 2–3 hrs/day)

You write every line of SQL yourself this week. I'll explain concepts, review your code, and point out bugs — but the CREATE TABLE, trigger, and procedure logic should be typed by you so you can defend it if asked in a viva/demo.

Day 1 (Today)

Finalize the ER diagram (you're basically done — just data types left)
Add data types to all columns in Lucidchart
Export final ER diagram as PNG/PDF for your report

Day 2

Set up MySQL/PostgreSQL locally (or a free cloud instance — Railway, Supabase, or ElephantSQL)
Write the full DDL script: CREATE TABLE for all 14 tables, with PK/FK constraints
Add CHECK constraints (status domain, role_type domain, city domain if not using FK)

Day 3

Add indexes: on all FK columns used in reports (route_id, trip_id, customer_id, driver_id, assistant_id, product_id, city_id)
Write and test 2–3 sample INSERT statements per table to confirm constraints work
Fix any FK/constraint errors that show up

Day 4

Write the 40-order data population script (10+ different routes, realistic delivery addresses, mixed customer types)
Populate PRODUCT, CUSTOMER, STORE, ROUTE, CITY, DRIVER, ASSISTANT, TRUCK reference data first (dependencies)

Day 5

Populate ORDER, ORDER_ITEM, ORDER_TRAVEL, TRAIN_SCHEDULE, TRUCK_DISPATCH (depends on Day 4 data)
Verify: query each table, confirm row counts and relationships look right

Day 6–7 (lighter days)

Write triggers: capacity deduction on TRAIN_SCHEDULE, driver/assistant double-booking prevention, 7-day delivery rule enforcement
Write 1–2 stored procedures (e.g., "assign order to next available trip if capacity exceeded")
Commit everything to Git (start a repo now if you haven't)

End of Week 1 milestone: Working database, fully populated, triggers/procedures functioning, schema documented.

WEEK 2 — Backend API

AI-assisted is fine here. I can generate most of this code directly — your job is to review it, run it against your real database, and understand it well enough to explain what each endpoint does.
Goals: REST API that exposes the core data and functions, connected to your real database.
Day 1–2: Project scaffolding (Node/Express or Django/Flask), connect to DB, set up `.env` config
Day 3–4: Build endpoints — customers, orders (create/read), products, routes, stores
Day 5: Build the report endpoints — quarterly sales, top items, sales by city/route, driver/assistant hours, truck usage
Day 6–7: Order placement endpoint that enforces the 7-day rule and capacity logic (calls your DB procedure), basic error handling
End of Week 2 milestone: Backend API running locally, all core endpoints tested (Postman/Thunder Client), reports return correct data.

WEEK 3 — Frontend

AI-assisted, kept basic. The brief only asks for a "basic UI" — don't over-invest here. A clean form for orders + tables/charts for the 6 reports is enough; skip elaborate design work.
Goals: Usable web interface for placing orders, viewing reports, and basic admin views.
Day 1–2: Project scaffolding (React/Vite), routing, connect to backend API
Day 3–4: Order placement form (customer selects product(s), address, route, delivery date) with 7-day validation
Day 5: Reports dashboard — quarterly sales, top items, city/route breakdown (tables/charts)
Day 6–7: Driver/assistant hours view, truck usage view, basic customer/order listing pages
End of Week 3 milestone: Frontend talks to backend, order placement and all 5 required reports render correctly.

WEEK 4 — Integration, Testing, Docs, Deployment

Day 1–2: End-to-end testing — place real orders through the UI, confirm DB triggers fire correctly, fix bugs
Day 3: Edge case testing (capacity exceeded → next trip, driver over 40 hrs rejected, assistant 3 consecutive rejected)
Day 4: Write project documentation — ER diagram, schema explanation, assumptions made, ACID justification (which triggers/procedures protect what)
Day 5: Deploy (Vercel/Netlify for frontend, Railway/Render for backend + DB) — optional but strong bonus if required
Day 6–7: Buffer for delays, polish UI, final report writeup, prepare demo/presentation if needed
End of Week 4 milestone: Fully working, deployed (or at least demoable), documented project ready for submission.

Notes

Scope is now confirmed: DB (manual, deep understanding) + basic backend + basic frontend. Weeks 2–3 stay light — don't gold-plate the UI.

Keep a running "assumptions" doc from Day 1 (product dimensions, route frequency, etc. — the brief explicitly asks for this).

Use Git from Day 1, commit daily.

Your ER diagram is finalized and confirmed to cover all brief requirements — no more structural changes needed, just finish adding data types.